

AI Trip Planner Review Guide

Presentation and reviewer preparation for two people

This guide prepares two presenters to explain, demonstrate, and defend the implemented AI Trip Planner. It also gives reviewers a practical inspection route, questions to ask, and evidence to request. The central result is a persistent travel-planning application that coordinates specialist agents and validates itinerary revisions as conditions change.

Review baseline: repository version bf85fdd, including Gemini support. Evidence recorded on 23 September 2026. Presenter 1 and Presenter 2 are speaking roles; replace those labels with your names during rehearsal. They do not imply who originally wrote each part of the code.

The project in one sentence

We extended an existing FastAPI and Next.js trip planner into a single-user local application with live-data integrations, persistent trips, explicit budget uncertainty, and automatic validated replanning, with a local model or Gemini for conversation.

Equal presentation ownership

Segment	Owner	Time	Main responsibility
Problem and approach	Presenter 1	0 to 2 min	Explain the travel-planning problem and project scope
Architecture and conversation	Presenter 1	2 to 4 min	Explain coordination, the interface, and LLM boundaries
Planning and adaptation	Presenter 2	4 to 6 min	Explain agents, budgets, and protected activities
Verification and limits	Presenter 2	6 to 8 min	Explain evidence, provider failures, and limitations
Shared demonstration	Both	8 to 12 min	Each leads approximately two minutes
Questions	Both	After 12 min	Primary owner answers; partner adds one useful detail

How to use this guide

Pages 2 and 7 explain the architecture and a worked example. Pages 3 and 4 provide speaking scripts. Page 5 is the demonstration runbook. Page 6 is the reviewer inspection guide. Pages 8 to 10 contain questions, suggested answers, and harder follow-ups. Page 11 contains the evidence index and final rehearsal checklist.

Presentation rule: describe current behavior precisely. Monitoring is periodic, traffic requires a configured provider, and unknown costs remain unknown. Do not claim a trained reinforcement-learning model, guaranteed optimal routes, booking functionality, or measured superiority over other systems.

Architecture and agent responsibilities

The request path

The Next.js interface sends a request through its backend proxy to FastAPI. The trip service validates the request and coordinates planning, persistence, and revisions. The orchestrator gathers provider data, invokes specialist agents, and passes the candidate schedule through the independent validator. SQLite stores the resulting trip and relevant revision history.

Planning flow: preferences and provider facts -> specialist agents -> candidate schedule -> validation -> saved trip -> interface.

Conversation flow: message and relevant trip context -> selected LLM -> validated preference changes -> the same planning flow. The LLM cannot directly insert a price or an arbitrary place into the plan.

Agent or component	Responsibility	Important boundary
Recommendation	Rank discovered attractions using TF-IDF and cosine similarity	Content matching, not a newly trained recommender
Restaurant	Rank listed dietary options	Dietary suitability may remain unknown
Accommodation	Fetch optional dated Amadeus offers	Listings alone do not establish price or availability
Weather	Retrieve forecasts and flag outdoor conflicts	Forecast coverage is limited
Transportation and traffic	Obtain walking or driving routes and traffic information	Geographic estimates are explicitly labeled
Emergency information	List source-provided nearby facilities and contacts	Information and directions only
Budget	Combine quotes, allowances, and spending	Incomplete totals cannot prove affordability
Itinerary	Schedule visits and travel gaps while retaining protected items	A bounded heuristic, not a global optimum
Orchestrator and validator	Coordinate agents and reject invalid revisions	Validation can fail and preserve the previous plan

Why call this a multi agent system

Agents have distinct responsibilities and use a shared interface under one orchestrator. The current implementation uses specialist software modules, provider adapters, and deterministic scheduling. It does not run a separate LLM for every agent, nor does it implement distributed agent negotiation or autonomous learning. This distinction is a useful answer if a reviewer challenges the terminology.

Code entry points: `api/main.py`, `services/trip_service.py`, `agents/live_orchestrator.py`, `agents/live_agents.py`, and `validation/live_validator.py`.

Presenter 1 speaking script

Owner: problem, scope, architecture, user experience, and conversational AI. Aim for four minutes, including pointing to the relevant parts of the interface. Speak naturally rather than reading every sentence verbatim.

Opening and problem

"Good morning. Our project is an AI Trip Planner based on a multi-agent architecture. A traveler often manages attractions, accommodation, weather, transport, and spending in different applications. Even a useful initial itinerary can become inconvenient when conditions change. Our goal is to bring these decisions into one persistent plan that can be reviewed and updated during the trip."

"Travel research covers areas such as attraction recommendation, itinerary optimization, and preference prediction. Our engineering focus is the coordination of these functions with current provider information and explicit constraints. We are presenting the implemented system and its test evidence; we are not claiming that every existing travel system is static or that we have outperformed all research methods."

Scope and user experience

"The user enters a destination, dates, traveler and room counts, interests, pace, transport mode, dietary preferences, and optional budget allowances. The application keeps saved trips, daily activities, expenses, conversations, and revisions. A user can lock an activity, mark it complete, record a commitment, pause monitoring, or request a refresh."

"This version is a single-user local application. Transportation is within the destination. Flights, payments, accounts, and booking transactions are outside the scope. Hotel and emergency information help the user make decisions, but the application does not reserve a room or dispatch assistance."

Architecture and the AI boundary

"The interface uses Next.js and TypeScript. FastAPI provides the backend API. A trip service coordinates the agent orchestrator, a separate validation layer, and SQLite storage. Each specialist agent has a defined task, so provider integration and scheduling logic remain inspectable. We extended the upstream repository and preserved its existing interfaces and history."

"For conversation, the user can select an existing local model or Google Gemini. The model extracts permitted preference changes and produces an explanation using supplied context. We validate its structured response before replanning. Invented place identifiers, unsupported fields, and monetary edits from the model are rejected. The form remains usable when the model is unavailable."

"Local mode uses a loopback endpoint. Gemini is an explicit cloud option: it sends messages and relevant trip context to Google, while its key stays in the backend configuration. The application does not silently switch from local inference to Gemini."

Handoff to Presenter 2

"I have explained how the user provides preferences and how the system is organized. My teammate will now explain how those preferences become a schedule, how the plan adapts, and how we verify that a revision is acceptable."

Be ready to show: the trip form, assistant setup guidance, saved trips, and the architecture on page 2. If asked about the research survey, cite only papers in the team's verified bibliography and distinguish their results from this application's results.

Presenter 2 speaking script

Owner: provider integrations, scheduling, budgets, persistence, monitoring, validation, and testing. Aim for four minutes before the shared demonstration.

Planning from provider data

"The planning agents use Open-Meteo for destination lookup and weather, and OpenStreetMap through Overpass for nearby attractions, restaurants, lodging, and emergency facilities. Amadeus hotel offers and TomTom routing are optional integrations that require credentials. Provider results carry availability and retrieval information so the interface can explain what is current, missing, cached, or estimated."

"The recommendation agent uses TF-IDF and cosine similarity to rank discovered places against the user's interests. The scheduler then considers the selected pace, available opening information, weather, and travel gaps. It protects locked, completed, committed, and already-started activities. These are heuristic decisions with independent validation, not proof of a globally optimal itinerary."

Budget and uncertainty

"Budget allowances are totals for the whole trip and all travelers. A usable hotel quote can provide accommodation cost; otherwise the planner uses the entered allowance. Other categories use allowances. Recorded spending replaces the covered part of a plan rather than being added twice. For example, an INR 3,000 food allowance and INR 1,000 already spent still project INR 3,000 for food."

"Blank and zero have different meanings. Blank means unknown, while zero is an explicit allowance. If a category is unknown, the total stays incomplete. If known costs already exceed the budget, the system reports a conflict instead of reducing the allowance silently. INR and USD conversion requires an explicitly configured rate, which is labeled as non-live."

Automatic updates and persistence

"The backend checks eligible trips every fifteen minutes while it runs. The visible browser refreshes its saved view every thirty seconds. Changed weather, opening information, route times, hotel offers, expenses, or preferences can lead to reevaluation. Only a validated candidate replaces the itinerary. If bounded replanning fails, the previous schedule is retained and the unresolved conflict remains visible."

"SQLite stores the trip and before-and-after revisions. Expected-version checks prevent an older monitoring result from overwriting a newer user edit. Due monitoring resumes after backend restart. The design currently assumes one backend worker for this local application."

Evidence and limits

"The recorded verification includes 192 backend tests, 30 frontend tests, lint, type checking, and a production build. The browser acceptance flow used isolated fixtures and a temporary database. Live Open-Meteo checks succeeded. Overpass failed during the recorded live checks, and successful live model, hotel, and traffic calls were not verified without their required configuration. We keep those evidence categories separate."

Handoff into the demonstration

"We will now show the user journey, then demonstrate how spending, protected activities, and revision history make changes reviewable. If an external provider is unavailable today, we will show its actual status and explain the tested behavior without presenting fixtures as live data."

Shared demonstration runbook

Before the review

Start the normal backend with `./run.ps1 backend` and the frontend with `./run.ps1 frontend` from separate PowerShell terminals. Open `http://127.0.0.1:3000/plan`. Verify provider status and, if conversation is part of the live demo, verify the configured model in advance. Never display the contents of a file containing API keys.

Choose dates within the available forecast window. Prepare an example destination with accessible provider data. Use demonstration preferences rather than personal travel details. Keep the test report available if a live dependency fails.

Step	Owner	Action	What to explain
1	Presenter 1	Enter destination, dates, pace, group size, and budget	These preferences become validated inputs
2	Presenter 1	Create the trip and inspect its daily itinerary and sources	Only discovered provider places enter planning
3	Presenter 1	Ask for a relaxed pace if a model is configured	The assistant proposes validated preference changes
4	Presenter 2	Lock one activity and record a food expense	User commitments and actual spending influence replanning
5	Presenter 2	Refresh and inspect the latest saved version	A refresh reevaluates conditions; it need not move activities
6	Presenter 2	Open History and inspect a recorded change	Explain the reason and before-and-after schedule
7	Presenter 1	Pause updates, then reopen the saved trip	Persistent data survives page navigation and reload

Timing: Presenter 1 leads steps 1 to 3 and 7 for about two minutes; Presenter 2 leads steps 4 to 6 for about two minutes. Rehearse the handoffs, not just the individual screens.

Demonstrating weather changes honestly

Natural weather changes are not guaranteed during a short review. For a repeatable demonstration, use the explicit tests/`browser_server.py` harness in a separate session with its temporary database. Label it on screen and verbally as a controlled test. Its weather-change endpoint advances the test clock and invokes the real monitoring path with fixture providers. The fixture LLM is also a test response, not live inference.

Prepare this separate session using `docs/review/README.md`. Restore the normal backend setting afterwards. Do not mix live and fixture results in one explanation.

If a dependency fails

Say: "This provider is unavailable in the current run. The application reports that condition. We can inspect the failure state now and use the explicitly labeled acceptance test to explain the successful revision path." Do not claim that pressing Refresh itself proves a weather-driven change or that estimated travel time is live traffic.

Reviewer inspection guide

The reviewer can use the following checks to connect each project claim to observable behavior. Ask the named presenter to lead the answer, then let the partner add implementation evidence where useful.

Review area	Lead	What the reviewer can say	Evidence to request
Problem and scope	Presenter 1	Explain which travel decisions you integrated and what is excluded	A form-to-saved-trip walkthrough and explicit boundaries
Multi agent design	Presenter 1	Show how agents are coordinated and exchange results	Orchestrator and BaseAgent interfaces; agent outputs
LLM reliability	Presenter 1	What prevents a generated price or invented place from entering the plan	Output contract and rejection tests
Provider provenance	Presenter 2	Which information is live, cached, estimated, or unavailable	Source status and retrieval times in the interface
Budget correctness	Presenter 2	Show why an unknown cost is different from zero	Blank allowance, recorded expense, and incomplete total
Revision safety	Presenter 2	Show what happens if a locked visit becomes infeasible	Retained schedule and unresolved conflict
Concurrent updates	Presenter 2	What happens when a user edits during background planning	Expected-version logic and conflict test
Evaluation quality	Both	Which results were live-tested and which used fixtures	Verification record and test files, separately identified

Suggested follow up sequence

Start with an observable claim: "You say the plan adapts. Show one change and the reason it was applied." Then probe the boundary: "What happens if there is no valid alternative?" Finally ask for evidence: "Where is that behavior tested?" This sequence checks understanding beyond a memorized feature list.

Useful review criteria

Assess whether the explanation matches the implementation, whether failures remain visible, whether the demonstrated revision preserves commitments, and whether test evidence supports the claim being made. Evaluate source transparency and reproducibility separately from visual polish. Do not treat the number of tests as a measure of recommendation accuracy.

Example reviewer feedback

"The separation between provider data, scheduling, and validation is clear. Strengthen the evaluation with successful live hotel and traffic checks, a documented comparison baseline, and user feedback on itinerary quality. State the present limitations of coverage and heuristic scheduling when presenting the results."

Presenter response: "Thank you. We will prioritize those evaluation gaps and keep them separate from already-tested functional behavior. We will report new measurements with their datasets, assumptions, and comparison method."

Worked examples for technical explanation

Budget accounting example

This is an illustrative calculation, not a provider quote or a saved sample trip. All values are INR totals for the trip and all travelers. The overall budget is INR 15,000.

Category	Planned allowance	Recorded spending	Projected category cost
Accommodation	8,000	0	8,000
Transportation	2,000	0	2,000
Food	3,000	1,000	3,000
Activities	1,500	0	1,500
Miscellaneous	Unknown	200	Unknown

For a known category, projected cost is $\max(\text{planned amount}, \text{recorded spending})$. The known lower bound is INR 14,700, including the INR 200 already spent in the unknown category. The full projected total remains unknown, so the application cannot confirm that this trip is within INR 15,000.

If the user explicitly sets miscellaneous to INR 500, the complete projection becomes INR 15,000. If food spending later reaches INR 3,600, food projects INR 3,600 and the overall projection becomes INR 15,600. The application reports an over-budget conflict. The allowance is a total budget for the category, not a remaining balance.

Weather revision example

Suppose a future itinerary contains an unlocked park visit and a locked museum visit. A new forecast flags the park day as unsuitable for outdoor activities. The scheduler searches for a feasible revision using discovered places, available hours, and travel gaps. The locked museum keeps its date and time. If validation passes, the service saves the new version and its explanation.

If the outdoor visit itself is locked and the weather conflict cannot be resolved while preserving it, the service retains the previous itinerary and reports the conflict. It does not silently unlock the activity or pretend the weather risk has disappeared. Completed and committed activities are also protected.

Concurrent edit example

A monitor starts from version 4. While it is planning, a user saves an edit that creates version 5. The monitor cannot save its result over version 5 because the SQLite save checks the expected version atomically. The stale write loses; the user's edit remains. This is optimistic concurrency control.

Who explains what: Presenter 2 walks through the calculation and revision. Presenter 1 points to the budget, activity controls, and history screens and explains the user-facing effect.

Questions led by Presenter 1

Why use multiple agents

Suggested answer: "The responsibilities differ. Place ranking, weather interpretation, budget calculation, and scheduling need different inputs and rules. Specialist modules make those decisions easier to test and inspect under one orchestrator."

Likely follow-up: Are they independent intelligent entities? "They are coordinated software agents with defined roles. This implementation does not claim distributed autonomy, negotiation, or separate LLMs for every agent."

What is the actual AI in the project

Suggested answer: "The LLM supports natural-language preference extraction and contextual explanations. Attraction ranking also uses content similarity through TF-IDF and cosine similarity. The scheduler and validation layer apply explicit constraints."

Likely follow-up: Did you train a model? "We did not train a new model or reinforcement-learning policy. We integrate a configured model and use the existing recommendation utilities."

Why use both local inference and Gemini

Suggested answer: "They offer different deployment choices. Local inference connects to an already running loopback server. Gemini is an explicit cloud option configured with a backend key and model identifier."

Likely follow-up: Is local mode completely offline? "Only inference is local. Destination lookup, discovery, forecasts, and optional routing or hotel offers still use external providers."

How do you prevent hallucinations

Suggested answer: "We validate structured output, permit only defined preference fields, reject unsupported place references, and block monetary changes from model output. Agents use provider facts when constructing the plan."

Likely follow-up: Can the explanation still be wrong? "Yes. These checks protect structured actions, but they do not prove that every sentence of model-generated prose is true. Source details and the validated itinerary remain the evidence to inspect."

Where is the Gemini key stored

Suggested answer: "It is read from backend configuration, normally the Git-ignored root .env file. It is not returned by the capability API or embedded in the frontend."

Likely follow-up: Is it encrypted at rest? "The .env file is not an encrypted secret store. This is a local single-user design; operating-system file access matters. A deployed system would need dedicated secret management and access controls."

What is the contribution beyond the upstream repository

Suggested answer: "We extended the existing architecture with persistent trips, live providers, additional specialist roles, explicit cost uncertainty, protected automatic revisions, a complete interface, and optional Gemini conversation. Upstream history and reusable interfaces remain."

Likely follow-up: Is this a new research algorithm? "The present contribution is the integrated system and its validated workflow. Claims of algorithmic novelty or better recommendation quality would require a separate comparative evaluation."

Questions led by Presenter 2

How does the system choose an itinerary

Suggested answer: "It ranks discovered attractions by content similarity, then tries time slots using pace, opening information, weather, and travel gaps. It keeps protected activities and checks the candidate independently."

Likely follow-up: Is the result optimal? "No global optimum is guaranteed. It is a bounded heuristic; reducing density can help resolve a conflict, and an infeasible revision is rejected."

What does real time mean here

Suggested answer: "The backend periodically refreshes provider information for eligible trips, by default every fifteen minutes. The visible interface retrieves saved updates every thirty seconds. Provider cache lifetimes and availability affect freshness."

Likely follow-up: What happens after restart? "Trips and due-check times are stored in SQLite. Monitoring resumes when the backend restarts. Nothing monitors while the backend is stopped."

How are costs and currencies handled

Suggested answer: "Costs come from usable hotel quotes, traveler allowances, and recorded spending. Unknown costs stay unknown. Currency conversion between INR and USD requires an explicit configured rate and is labeled as non-live."

Likely follow-up: Why not add expenses to the allowance? "The allowance covers the whole trip category. Adding all spending again would double-count it. We use the larger of the planned total and actual spending."

What happens when providers fail

Suggested answer: "Requests use timeouts, bounded retries, caches, and rate-limit handling. Failures and missing credentials remain visible. Route estimates can be shown with explicit labels, but missing places are never replaced with sample attractions."

Likely follow-up: Can a trip still be created? "It can be saved with unavailable information and conflicts. A useful validated itinerary still depends on adequate discovered places and feasible constraints."

What protects a user commitment

Suggested answer: "The scheduler retains locked, completed, committed, and already-started activities. Validation rejects changes that violate protected constraints, and a failed replan retains the previous schedule with its conflict."

Likely follow-up: Does a commitment mean a real booking? "No. It is a scheduling protection selected by the user. The application performs no booking transaction."

Why use SQLite and version numbers

Suggested answer: "SQLite fits this single-user local application and persists trips without a separate database server. Expected-version checks inside a transaction stop stale requests from overwriting newer edits."

Likely follow-up: Would this support many simultaneous users? "That is outside the present design. Accounts, authorization, stronger deployment controls, and worker coordination would need additional engineering and testing."

Harder follow up questions and closing

What do the test results prove

Lead Presenter 2: "The recorded 192 backend tests and 30 frontend tests verify selected functional contracts and regressions. Lint, type checks, and the production build also passed. These results do not measure recommendation accuracy, user satisfaction, or superiority over another travel planner."

Follow-up: Which live integrations passed? "Open-Meteo lookup and weather succeeded in the recorded checks. Overpass failed in that environment. Successful live LLM, Amadeus, and TomTom calls were not verified without their required configuration."

How can you demonstrate adaptation if the provider is down

Lead Presenter 2: "We show the current failure state honestly. A separate, explicitly labeled acceptance harness can demonstrate the monitoring and validation workflow with controlled inputs and a temporary database. That is functional test evidence, not a successful live-provider demonstration."

Follow-up: Why use controlled inputs? "They let us repeat a specific weather conflict, assert what must stay fixed, and inspect the saved revision without waiting for nature or network conditions to change."

Where is your comparison with the twenty research papers

Lead Presenter 1: "The literature survey should identify each paper, its task, data, method, evaluation, and limitation. Our implementation review demonstrates what this system does. A claim that it performs better needs comparable datasets and metrics, which are separate from the current functional tests."

Follow-up: What should the comparison evaluate? "Relevant candidates include constraint violations, feasible-itinerary rate, response time, preference relevance, change disruption, and user satisfaction. These are proposed evaluation measures, not reported results."

Can every date and destination be supported

Lead Presenter 1: "No. Place discovery depends on OSM coverage and the configured search area. Forecasts have a limited horizon; opening rules can be unknown or unsupported. The application exposes those limits instead of guaranteeing worldwide coverage or ticket availability."

Follow-up: What is the current discovery scope? Presenter 2 adds: "The implementation searches within six kilometres of the resolved city centre and caps the OSM response at 400 elements. It does not exhaustively discover an entire city."

What would you improve next

Lead Presenter 2: "First, complete credentialed live checks and broaden provider coverage. Then evaluate itinerary quality against a documented baseline, improve opening-hours interpretation and route constraints, and measure whether revisions are useful to users."

Presenter 1 adds: "A later version could improve preference explanations and accessibility. Multi-user deployment, bookings, and a learned planning policy would be separate projects, not features we claim today."

Suggested closing

Presenter 1: "The system brings planning preferences, provider information, and saved trips into one interface."

Presenter 2: "Its key behavior is that changes remain constrained and reviewable: valid revisions are saved, commitments are protected, and unresolved information is visible. We welcome questions about the implementation and its evidence."

Evidence index and rehearsal checklist

Repository evidence to keep ready

Evidence	Where to inspect	Lead
Setup and provider requirements	README.md and docs/GUIDE.md	Presenter 1
UI and conversational flow	frontend/components/TripDashboard.tsx	Presenter 1
LLM validation and Gemini configuration	services/local_llm.py and tests/test_gemini.py	Presenter 1
API and persisted trip contracts	api/main.py and models/live.py	Presenter 2
Scheduling and orchestration	agents/live_agents.py and agents/live_orchestrator.py	Presenter 2
Revision and concurrency logic	services/trip_service.py and services/trip_store.py	Presenter 2
Independent itinerary checks	validation/live_validator.py	Presenter 2
Monitoring and provider tests	tests/test_monitoring.py and tests/test_live_planning.py	Presenter 2
Recorded verification and browser harness	VERIFICATION.md and tests/browser_server.py	Both

Facts both presenters should remember

Timing: 15-minute backend checks, 30-second visible browser refresh, and three default replanning attempts. Monitoring covers ongoing trips and trips starting within 15 days while the backend runs.

Evidence: 192 backend tests, 30 frontend tests, lint, type checking, and production build passed in the recorded verification. Unknown prices and unavailable traffic remain explicit. Local inference and Gemini are selected deliberately, with no automatic cloud fallback.

Final rehearsal

Agree on names and rehearse the twelve-minute flow with providers available and unavailable. Practice the budget calculation, handoffs, and locating one supporting test each. Keep secrets off screen. Answer the question first, then offer evidence.

Sources and version

Baseline: application bf85fdd and the verification record dated 23 September 2026. The files above support the implementation claims. Budget and weather examples are illustrations, not measured trip results.

[Project repository](#) | [Reviewed code version](#) | [Verification record](#)

Provider references are in the configuration guide. Cite the team's verified bibliography separately for literature-review claims.